

CS 526: CoS-SSA Final Report

Alex Broihier and Devansh Jain

May 15, 2025

1 Introduction

Compilers rely on Static Single Assignment (SSA) [1, 2] form to simplify dataflow analyses by ensuring each variable has exactly one definition. This transformation collapses complex many-to-many def-use chains into straightforward one-to-many relationships, where each use of a variable directly references a unique definition. This enables sparse dataflow analysis, which propagates information only along the SSA graph’s def-use chains rather than across all control-flow edges. As a result, analyses such as constant propagation, dead code elimination, and register allocation become more efficient. For instance, sparse conditional constant propagation combines constant propagation and dead code elimination in a single pass, leveraging SSA’s structure to identify and remove unreachable code paths effectively.

However, mainstream SSA implementations in LLVM and GCC remain intraprocedural for memory operations: global variables are treated as ordinary loads and stores, never renamed or merged via ϕ -functions [3]. As a result, interprocedural optimizations such as constant propagation, dead-code elimination, and value numbering must conservatively approximate global-variable behavior across function calls, losing precision whenever a global is modified in multiple contexts [4, 5]. For example, LLVM’s `-globalopt` and Interprocedural Sparse Conditional Constant Propagation (ISCCP) passes operate context-insensitively and cannot propagate constants through repeated, differently-contextualized updates of a global counter [6]. This conservative treatment leads to missed optimization opportunities and suboptimal code generation in codebases with pervasive global state.

Context-Sensitive SSA (CoS-SSA) introduces context-sensitive metamorphic assignments for globals where each global write is renamed per calling context. A top-down, fixed-point construction algorithm performs interprocedural, value-based reaching definitions analysis to build a fully context-sensitive and flow-sensitive dataflow graph for globals [7]. Integrating CoS-SSA into LLVM promises to unlock finer-grained global-variable optimizations, improving both static analysis precision and final code quality.

2 Prior Work

2.1 Intraprocedural Extensions

Extensions such as Heap Array SSA [8], Region Array SSA [9], and Memory SSA [3] incorporate pointer-induced memory operations into SSA by introducing memory ϕ -nodes to merge may-reach definitions, enabling sparse intraprocedural analyses of loads and stores.

2.2 Interprocedural Memory SSA

Interprocedural Memory SSA (IMSSA) extends Memory SSA across function boundaries, modeling indirect memory operations globally; however, its construction is costly due to pointer aliasing and whole-program analysis [4]. Parallel SSA construction techniques partition memory objects and exploit multi-core resources to speed up IMSSA building by up to $6.9\times$ on large codebases [4]. SVF (Static Value-Flow Analysis) in LLVM constructs an interprocedural value-flow graph for scalars and address-taken variables, iteratively refining pointer analysis and Memory SSA, yet remains context-insensitive for globals [5].

2.3 Global-Variable Optimization in LLVM

LLVM’s `-globalopt` and `-globaldce` passes optimize globals by marking non-address-taken variables constant and eliminating dead stores, while ISCCP propagates constants interprocedurally but aggregates effects from all call-sites conservatively [6]. These analyses cannot precisely handle code where the same global is updated differently in distinct call contexts, leading to lost optimization opportunities, such as repeated global increments inside looped calls.

2.4 Context-Sensitive SSA (CoS-SSA)

CoS-SSA [7] augments SSA with context-sensitive metamorphic assignments for global variables that distinguish definitions per call strings. This representation preserves full flow and context sensitivity, enabling precise interprocedural reaching definitions for globals. While the theoretical algorithm is detailed in [7], its integration into LLVM remains unexplored.

3 High Level Overview

Our CoS-SSA implementation is built as a standalone LLVM IR transformation that operates independently of prior analyses, enabling broad applicability to any LLVM-based programming model. The system is structured into five main passes as shown in Figure 1.

1. Extract CFG: Translates LLVM IR into a simplified control flow graph (CFG) tailored for our analysis. This includes identifying global variable definitions/uses, modeling external interactions abstractly, and partitioning globals into non-interacting sets to ensure scalability.
2. Compute DFG: Constructs a context-sensitive dataflow graph (DFG) by propagating qualified definitions (QDefs) throughout the CFG using the fixed-point algorithm detailed in [7].
3. Optimize DFG: Performs constant propagation, merges duplicate qualified definitions, and eliminates dead global definitions.
4. Generate CoS-SSA: Transforms the optimized DFG into a context-sensitive SSA form. Assignments are rewritten as metamorphic where needed, and ϕ -nodes are inserted naively at uses with multiple reaching definitions.
5. Destroy CoS-SSA: Converts the CoS-SSA representation back into valid LLVM IR, using if-else branches to handle context-sensitive behavior.

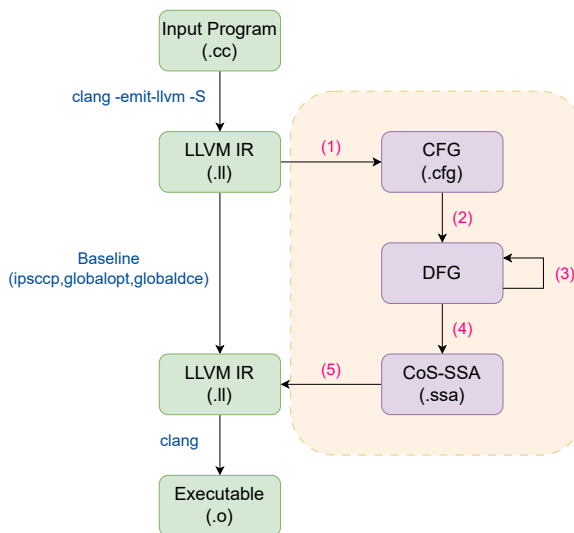


Figure 1: Overview of CoS-SSA implementation

4 Implementation Details

4.1 Prior Analyses or Transformations

Our CoS-SSA implementation, using version 15.0.7 of the LLVM API, takes standalone LLVM IR files as input and outputs modified versions of the LLVM IR files. Thus, it does not rely on prior analyses or transformations to the code other than the code being lowered to LLVM IR; this opens up the opportunity to apply CoS-SSA to any program that is lowered to LLVM IR as an intermediate step.

4.2 Major Code Components

Our CoS-SSA implementation operates in five broad passes: Extract CFG, Compute DFG, Optimize DFG, Generate CoS-SSA, and Destroy CoS-SSA.

(1) Extract CFG. We first extract the CFG from the LLVM IR into our own CFG data structure. We do this, rather than write all of our algorithms over the LLVM IR directly, because it allows us to represent the code in a simpler format, allowing for simpler implementations of the following algorithms. In addition, this allows us to easily “promote” code features to global variables; we use this to treat return instructions as writes to a global for that function (a “return variable”), and uses of the returned value as reads from that variable. As we extract the CFG, we record all definitions and uses, arithmetic expressions they are involved in, and function calls as nodes in our new CFG. We do not deal with alias analysis and thus throw out any global that has its address used in a way other than loading or storing variables to that address.

As we abstract the CFG, some globals may interact with parts that we do not abstract (for instance, a global being passed to *printf*). We model these external uses as an assignment to a *USEVAR* global. Similarly, we model external dependencies with an *INPUT* global. This allows our following algorithms to take place without breaking interactions with the surrounding code that is not represented in our CFG.

Finally, we partition global variables into limited-size sets. We do this to limit the maximum number of globals the following algorithms have to process at once. Since the algorithms are context-sensitive, they would otherwise not scale to large programs; this is detailed in our evaluation. Global variables are put into sets based on how many times they interact with (use/are used by) other variables in the same set. A disjoint set data structure represents these partitions. Variables in different partitions may not interact with each other; if necessary, *USEVAR* and *INPUT* are used to split an assignment to keep partitions separate. For instance, if a and b belong to different partitions, $a = b$ is split into $USEVAR = b$ and $a = INPUT$.

(2) Compute DFG. To compute the DFG, we create a worklist implementation that performs a top-down forward dataflow analysis over the CFG based on fixed-point equations detailed in [7]. This algorithm computes which qualified definitions are live at each node (for each context at that node) in the CFG. Upon reaching function calls, new contexts are created based on the qualified definitions that reach the call site. We use these value-based contexts rather than call-site-based contexts. Prior work like VASCO [10] has shown that value-based contexts are as precise as arbitrary-length call strings. Unlike value-based contexts, call strings can run into problems with recursive calls, leading to unbounded length strings with approximations like k -length call strings. And thus, we use value-based contexts to maintain full context-sensitivity. We store these value contexts in a context table data structure, analogous to the symbol tables common in compilers.

(3) Optimize DFG. Using the DFG, we first do constant propagation (and some constant folding) with a simple worklist algorithm; if the inputs for a qualified definition all have known values (and these known values are consistent across different versions of the same variable), then we compute the assignment value for the qualified definition. Next, we implement an optimization specific to CoS-SSA: we merge duplicate qualified definitions to reduce the total number of contexts. We repeat this process, using the propagated values wherever possible, until no more assignments can be reduced. Finally, we implement dead code elimination. All qualified definitions are marked dead. Then, qualified definitions assigned to *USEVAR* are marked live. Finally, all qualified definitions transitively used by live qualified definitions are marked live.

The constant propagation and dead code elimination are easy to implement over the DFG. Furthermore, other SSA-based optimizations can be developed for global variables using the context-sensitive DFG, allowing for more precise interprocedural analysis.

(4) Generate CoS-SSA. We use the optimized DFG to create an SSA version of our CFG graph. This primarily consists of transforming regular assignments into metamorphic assignments when an assignment is involved in more than one context. The optimizations computed on the DFG are applied as well: the right-hand side of assignments is replaced with a constant in contexts where the value is known, and dead code is removed from the resulting graph. Finally, when more than one version of a variable reaches a use in a single context, we take a naive approach and insert classical ϕ -nodes right before the use. There is a lot of existing work on optimizing ϕ -node placements, with efficient implementations computing dominance

frontiers; these existing approaches could be applied to better optimize our ϕ -node placement. However, we will see in Destroy CoS-SSA that there is no benefit to optimizing the placement of ϕ -nodes in our implementation.

Note that we use the DFG to construct CoS-SSA; the DFG can be easily recovered from the CoS-SSA and used to implement further optimizations on CoS-SSA. We omit code to recover the DFG from CoS-SSA since our passes do not perform further optimizations once the final CoS-SSA is constructed.

(5) Destroy CoS-SSA. Finally, we destroy the CoS-SSA by using it to update the original LLVM IR. For each partition of global variables, a new global is introduced to maintain the current context. Around function calls, if the callee contains metamorphic assignments, the old context is stored in a virtual register, the new context is set through if-else branches on the old context, and finally, the old context is restored after the call. To deconstruct metamorphic assignments, if-else branches are constructed based on the current context, where each branch is the assignment for one context.

We also have to represent the CoS-SSA ϕ -nodes. CoS-SSA ϕ -nodes can take in global variables live from a different function call and thus do not work well with LLVM ϕ -nodes. Classical ϕ -nodes can be destroyed by creating a corresponding assignment at the end of each incoming basic block; we adopt a similar methodology. CoS-SSA ϕ variables are assigned when the incoming variable is assigned. To keep from ballooning in the number of variables, we do not create duplicate versions of global variables, but reuse the same variable.

Finally, return values that were “promoted” to global variables are destroyed into the respective return statements and uses of the value returned from a function call.

In our approach, we choose to represent most of the known information of the CoS-SSA in the LLVM IR; more advanced destruction algorithms may chose to destroy the CoS-SSA in a different way (such as by creating duplicate versions of functions a limited number of times) or may use heuristics to determine when introducing additional contexts is worth it.

4.3 Testing Strategy and Status.

We start with 15 unit tests to test Create DFG, Optimize DFG, and Generate CoS-SSA. These unit tests start with a hand-designed CFG curated to test a wide range of control flow features, including loops and recursion. We manually inspect the input CFG and the final CoS-SSA visualized using Graphviz.

We then have 9 C files that serve as unit tests for the entire program. These tests check for correct handling of different CoS-SSA constructs when creating an output LLVM IR file, and for different cases that can come up. We test: constant propagation, phi nodes, dead code elimination, input variables, metamorphic assignments, a simple return value, metamorphic return values, multiple global partitions, and our motivating example. These are automatically checked against expected outputs.

Finally, we looked for real-world programs, both from real-world benchmarks written in C and C++, and existing real-world applications. These programs were hard to find; a lot of code does not make use of many globals, and of code that does, oftentimes, globals are in the form of complex structs, which our implementation does not support. We settled on programs used in competitive programming (`enchantedmaze`), from the MiBench benchmark [11] (`dijkstra_large`), from a version of SQLite called SQLite Amalgamation [12] (`shell`), and four versions of a Tiny Basic interpreter [13] (`dds7`, `int-*`). These programs that we found are heavily interactive, and thus, performance benchmarking is difficult, unlike typical benchmark suites.

4.4 Implementation Status

All of the described functionality works as intended on all of our unit tests, both the CFG and end-to-end LLVM IR tests. Our tests/benchmarks generate no Valgrind warnings/errors.

Our implementation produces compilable outputs for all the small and large source programs some of the programs we tested. There are two cases for why a compilable output is not produced. (1) Some constants are propagated with the incorrect integer type. This is likely due to a bug in our DFG construction, because we caught this late, we have not been able to debug this yet. (2) We do not account for function pointers and interrupt handlers, and thus do not traverse the bodies of some functions when computing the DFG, incorrectly mark certain assignments as dead, and lead to incorrect IR when we destroy CoS-SSA.

5 Experimental Results

We used our implementation to perform context-sensitive optimizations for globals in 7 real-world C/C++ programs with several non-address-taken global variables. Table 1 summarizes statistics of the chosen benchmarks as well as resource utilization. The resource utilization statistics were recorded on a 20-core 13th Gen Intel(R) Core(TM) i7-13700H CPU with 32 GB RAM.

Observation: Only a small subset of global variables interact with each other, resulting in small partitions.

Observation: Partitioning of global variables is critical for the scalability of the CoS-SSA construction.

Benchmark	sloccount	#Procs	#Nodes	#Globals	#Partitions (max size)	Resource Utilization	
						w/o partitions	w/ partitions
dds7	352	15	287	7	7 (1)	0.39s / 68MB	0.02s / 10MB
dijkstra_large	134	6	81	7	6 (2)	0.14s / 19 MB	0.01s / 9MB
enchantedmaze	228	141	1103	100	56 (5)	0.88s / 192MB	0.09s / 19MB
int-stc	547	37	706	20	19 (2)	60s+ / 8GB+	0.61s / 31MB
int-stc2	620	37	724	20	19 (2)	60s+ / 8GB+	12.12s / 180MB
int-stcp	493	141	1339	60	40 (5)	1.17s / 134MB	0.05s / 15MB
shell	18094	347	8014	231	225 (3)	60s+ / 8GB+	0.68s / 30MB

Table 1: Resource utilization for CoS-SSA construction for different benchmarks.

Further, we analyzed the final CoS-SSA generated per global partition, as well as optimization opportunities discovered by the context-sensitive dataflow graph. These statistics are summarized in Table 2.

Observation: Context-sensitive analysis of global variables detected multiple optimization opportunities.

Benchmark	#Procs	#Globals	#Contexts		#QDefs		Constant Propagation	Dead Code Elimination
			(max)	(total)	(max)	(total)		
dds7	15	7	99	343	44	305	3	5
dijkstra_large	6	7	29	93	55	242	10	14
enchantedmaze	141	100	143	1126	259	10276	46	127
int-stc	37	20	123	694	168	1683	81	28
int-stc2	37	20	705	2682	298	4073	346	26
int-stcp	141	60	71	1139	79	2615	59	67
shell	347	231	10	682	19	2265	6	240

Table 2: Statistics of CoS-SSA and optimizations discovered. (max) refers to maximum per partition.

Baseline: LLVM passes related to global variable optimizations (`-globalopt`, `-globaldce`, `-ipsccp`) did not identify any optimization opportunity for global variables (confirmed using `diff` over the IR files).

6 Conclusion

We present an implementation of context-sensitive SSA (CoS-SSA). CoS-SSA allowed us to easily implement context-sensitive compiler optimizations on global variables that identify optimization opportunities not found by optimizing production compilers, namely Clang at the `-O3` optimization level. Despite being fully context-sensitive, partitioning global variables into small groups allows our CoS-SSA construction algorithm to scale well on most input programs, with a typical compilation overhead of less than a second. Finally, our CoS-SSA implementation shows promise at being an analysis tool that can either inform the output code or be ignored; future work can look into more effective ways of deconstructing CoS-SSA, perhaps by using heuristics to determine whether to use if-else branches or simply clone procedures.

References

- [1] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, “Efficiently computing static single assignment form and the control dependence graph,” ACM Trans. Program. Lang. Syst., vol. 13, p. 451–490, Oct. 1991.
- [2] B. K. Rosen, M. N. Wegman, and F. K. Zadeck, “Global value numbers and redundant computations,” in Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’88, (New York, NY, USA), p. 12–27, Association for Computing Machinery, 1988.
- [3] T. L. Project, “MemorySSA Documentation.” <https://llvm.org/docs/MemorySSA.html>, 2020.
- [4] Y. Sui et al., “Parallel construction of interprocedural Memory SSA form,” Journal of Systems Architecture, vol. 85, pp. 23–35, 2018.
- [5] Y. Sui and J. Xue, “SVF: interprocedural static value-flow analysis in LLVM,” in Proceedings of the 25th International Conference on Compiler Construction, CC ’16, (New York, NY, USA), p. 265–266, Association for Computing Machinery, 2016.
- [6] L. D. Meeting, “A Deep Dive into the Interprocedural Optimization Infrastructure.” https://llvm.org/devmtg/2020-09/slides/A_Deep_Dive_into_Interprocedural_Optimization.pdf, 2020.
- [7] D. Jain, “CoS-SSA: SSA for Context-Sensitive Interprocedural Analysis (attached in the tar),” B.Tech. Thesis Project Report, IIT Bombay, April 2023.
- [8] K. Knobe and V. Sarkar, “Array SSA form and its use in parallelization,” in Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’98, (New York, NY, USA), p. 107–120, Association for Computing Machinery, 1998.
- [9] S. Rus, G. He, C. Alias, and L. Rauchwerger, “Region array SSA,” in Proceedings of the 15th International Conference on Parallel Architectures and Compilation Techniques, PACT ’06, (New York, NY, USA), p. 43–52, Association for Computing Machinery, 2006.
- [10] R. Padhye and U. P. Khedker, “Interprocedural data flow analysis in Soot using value contexts,” in Proceedings of the 2nd ACM SIGPLAN International Workshop on State Of the Art in Java Program Analysis, SOAP ’13, (New York, NY, USA), p. 31–36, Association for Computing Machinery, 2013.
- [11] M. Guthaus et al., “MiBench: A free, commercially representative embedded benchmark suite.” <https://vhosts.eecs.umich.edu/mibench/>, 2001.
- [12] A. Zamani et al., “SQLite Amalgamation.” <https://github.com/azadkuh/sqlite-amalgamation>, 2022.
- [13] E. Davis, “Tiny Basic: Interpreters.” <https://github.com/eddavis2/Tiny-Basic/tree/main>, 2023.

A Appendix

We discuss two simple examples that show optimization opportunities enabled by context-sensitive SSA (CoS-SSA) construction but not detected by optimization passes in a production compiler like `clang -O3`.

A.1 Motivating Example (from Presentation)

```
1 #include <stdio.h>
2
3 int x, w, t;
4
5 void P() {
6     t = 50;
7     x = w + t;
8 }
9
10 int main() {
11     int INPUT, USEVAR;
12     if (INPUT) {
13         w = 10;
14         if (INPUT) {
15             P();
16             USEVAR = x;
17             printf("x = %d\n", USEVAR); // x = 60
18             return 0;
19         } else {
20             P();
21             USEVAR = x;
22             printf("x = %d\n", USEVAR); // x = 60
23         }
24     } else {
25         w = 20;
26         P();
27     }
28     USEVAR = x;
29     printf("x = %d\n", USEVAR); // x = 60 or 70
30     return 0;
31 }
```

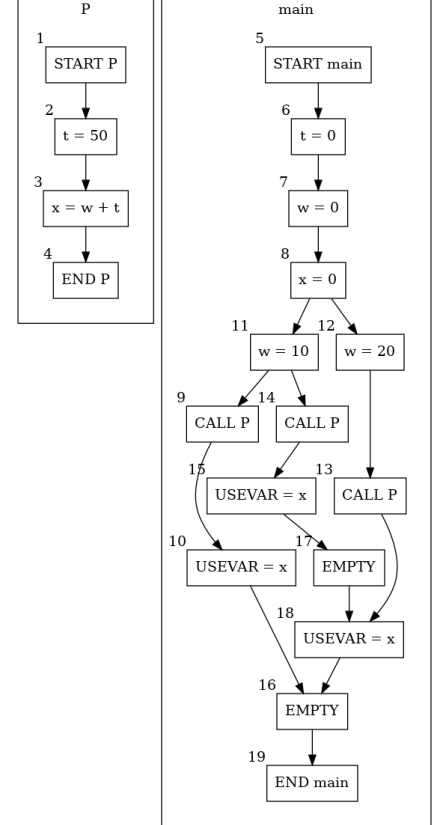


Figure 2: Motivating Example: Input C Program (left) and Extracted CFG (right)

Figure 2 shows a C program and the resulting CFG we extract from it. The CFG contains the minimum amount of information needed to just focus on global variables, and other uses of globals by non-global variables are abstracted with *USEVAR*. We can easily reason from the source code that *x* is a constant value (60) on lines 16 and 21. Next, we show how CoS-SSA allows context-sensitive analysis to enable constant propagation. Note that existing compiler passes on global variables are unable to perform this optimization.

Figure 3 is the resulting CoS-SSA; we show an unoptimized and optimized example. Variables are in the form of *x.a.b*, where *a* is the node the variable is defined at and *b* is the context it is defined in. There are two value contexts for procedure *P* represented as the numbers 2 and 3, and we see metamorphic assignments for *t* and *x*. We see that the assignment to *t* at node 2 can be merged into one assignment, or even removed if we replace all uses of *t* with 50. Node 18 has a ϕ node inserted because multiple versions of the variable *x* can reach node 18 in the same context.

In the optimized version, we remove all dead assignments after propagating the constants. Now we know that *x* = 60 for the two uses on lines 16 and 21 (nodes 10 and 15), and can determine that it is one of two values on line 28 (node 18).

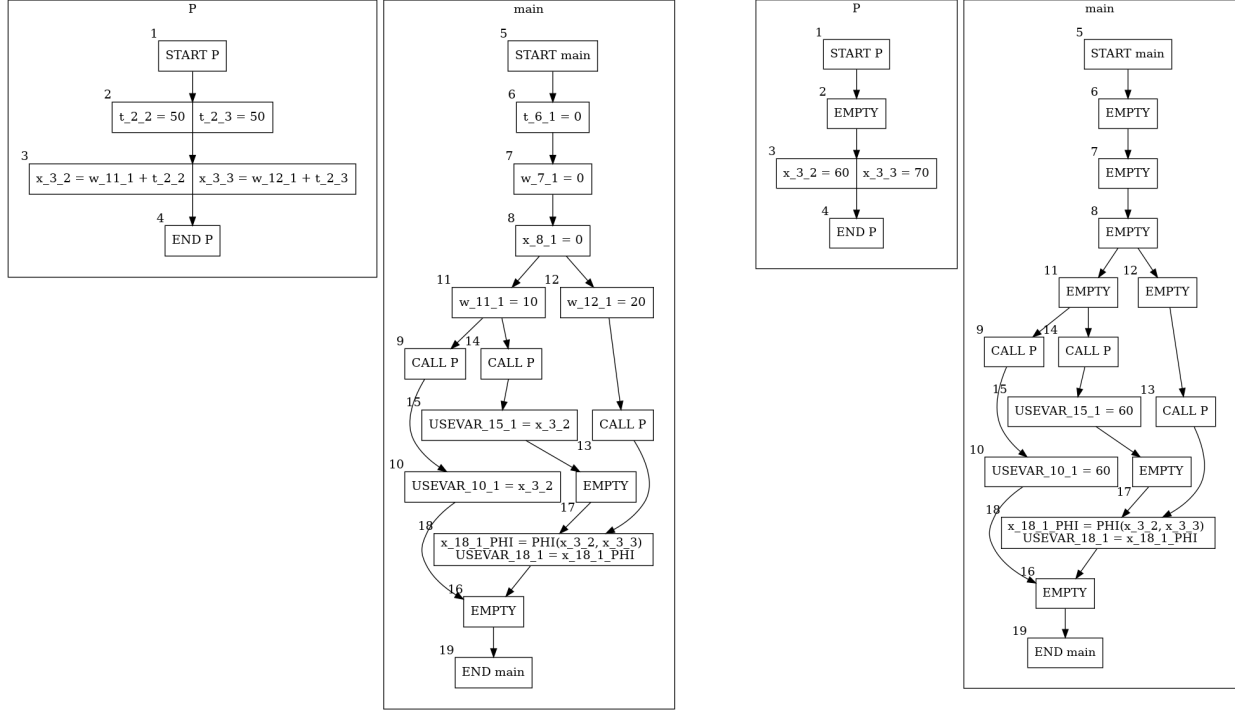


Figure 3: Motivating Example: CoS-SSA before (left) and after (right) optimizations

A.2 Lifting Return as Global Variable

```

1  #include <stdio.h>
2
3  int i;
4
5  int fn() {
6      int INPUT, USEVAR;
7      i = i + 20;
8      return i;
9  }
10
11 int main() {
12     int INPUT, USEVAR;
13     i = 0;
14     USEVAR = fn();
15     printf("%d\n", USEVAR);
16
17     return 0;
18 }

```

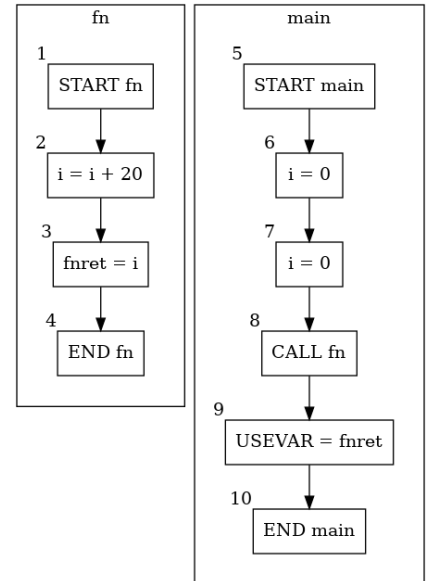


Figure 4: Lifting Return as Global Variable: Input C Program (left) and Extracted CFG (right)

Figure 4 shows a program where a function returns a value from the function fn . In our CFG, we “promote” fn ’s return value to a global variable. Then line 8 becomes $fnret = i$ and line 14 becomes $USEVAR = fnret$. Note that were we to lower this code back to LLVM IR, definitions of $fnret$ would be

converted back to return statements, and uses of *fnret* would be converted back to uses of *fn*'s return value.

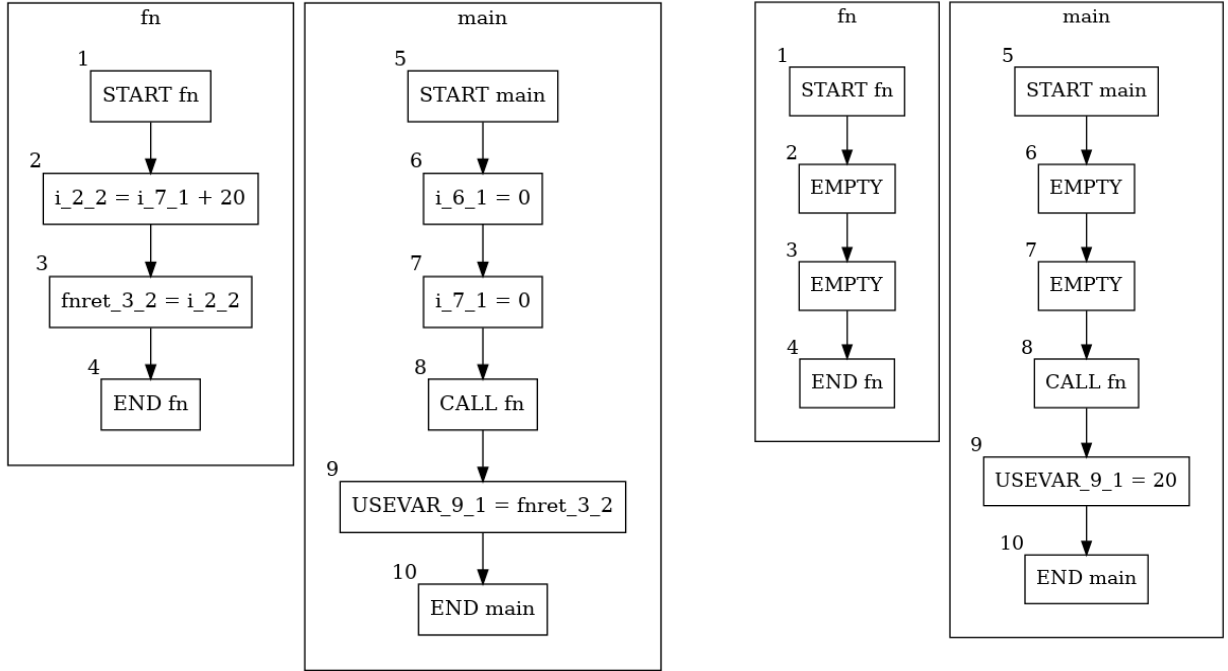


Figure 5: Lifting Return as Global Variable: CoS-SSA before (left) and after (right) optimizations

Figure 5 now shows us applying our regular CoS-SSA construction and optimizations with *fnret*, ultimately deriving that its one use has a value of 20 and that the body of *fn* can be removed. A subsequent, context-insensitive, compiler pass could then remove *fn* and the call to *fn*. This example shows how return values can be expressed in CoS-SSA, allowing for context-sensitive optimizations to be performed on them.